SECURE PROGRAMMING · BIBIFI PHASE II

Threat Model Report

Secure Gallery Tracker — "The Emperor's Collection"

A STRIDE threat model of a Flask / PostgreSQL gallery-visit tracking application, with risk prioritization and mitigation guidance.

Team	Wil & Charlie
Course	Secure Programming
Assignment	Threat Modeling — due 10/30
Repository	Secure-Programming-BIBIFI
Baseline	commit `f60f642`
Revision	Corrected against the repository source and schema

> **How to read the status column.** Implemented control exists in the repository; Partial control exists but does not fully cover the threat; Open no control exists; Planned feature is designed but not built. Risk ratings reflect the system as it exists today, not the intended end state.

Contents

1. System Overview
2. Assets
3. Threat Actors
4. System Decomposition and Data Flow Diagram
5. STRIDE Threat Identification
6. Risk Analysis and Prioritization
7. Mitigation Strategies
8. Threat Summary Table
9. Conclusion
10. References
11. Appendix A — As-Built vs. Target Design
12. Appendix B — Evidence Index

1. System Overview

Purpose. The system is a web application for an art gallery, "The Emperor's Collection", that records when signed-in users enter and leave individual rooms and shows who is currently in each room. It combines authentication, role-based authorization, and an audit trail so that movement records and administrative actions can be attributed. Gallery-level visit tracking and administrative user management are part of the intended design but are *not yet implemented*; they are marked **Planned** throughout this report.

Primary functionality

- **Login and session management** — Argon2id verification, lockout, server-side sessions, logout revocation. **Implemented**
- **Room enter/leave events** — `POST /rooms/<room_id>/visit` records an `enter` or `leave` event for a room. **Implemented**
- **Room and artwork browsing** — home page, room list, room detail, current visitors. **Implemented** (the `/rooms` list route currently renders a missing template)
- **Personal movement history** — a per-user history view. **Planned**
- **Gallery-level enter/leave** and the "must be in the gallery before a room" state machine. **Planned**
- **Administrative user and role management** — `/admin` currently returns a JSON placeholder. **Planned**
- **Audit of security-relevant actions** — login and logout only. **Partial**

Architecture

```
Browser  --HTTP on 127.0.0.1:8000 locally / HTTPS when deployed-->  web container
                                                                   Gunicorn, Flask, 3 workers
                                                                          |
                                                                          | internal Docker network only
                                                                          v
                                                                   db container
                                                                   PostgreSQL 16, no host ports
```

Local development serves plain HTTP; TLS is expected to terminate at a reverse proxy in deployment. `FORCE_HTTPS=false` by default, which also disables the cookie `Secure` flag and HSTS (`auth.py:149` , `docker-compose.yml:31`).

Technologies

Concern	Choice
Language / framework	Python 3.12, Flask 3.0.3, Jinja2 auto-escaping
Server / packaging	Gunicorn (3 workers) in Docker; Docker Compose
Database	PostgreSQL 16; SQLAlchemy Core with psycopg 3, parameterized SQL
Authentication	Flask-Login, argon2-cffi (Argon2id), hashed server-side sessions
Request protection	Flask-WTF CSRFProtect, Flask-Limiter, Flask-Talisman (CSP, HSTS when enabled)
Configuration	Git-ignored `.env` : DB credentials, `SECRET_KEY` , `FORCE_HTTPS`
Testing	pytest (`test/security/test_auth.py`); the suite currently cannot run because `fakes.py` is missing

Roles

Role	Permissions in the design	Status
Guest	View rooms and artworks; record and view only their own room movements.	**Partial** — movements are self-recorded; "own history" is not built.

Role	Permissions in the design	Status
Employee	Record movements, including on behalf of another account; view gallery occupancy.	**Implemented** — delegated recording exists at `app.py:83-89`.
Administrator	Manage users and roles; view the full audit log.	**Planned** — route placeholder only.

2. Assets

Assets are ranked by the damage that loss of confidentiality, integrity, or availability would cause. The identifiers are used in the threat tables.

ID	Asset	Why it matters	Primary property
A1	User credentials (Argon2id hashes)	Offline cracking or reuse leads to account takeover; hashes must never leave the DB.	Confidentiality, Integrity
A2	Flask `SECRET_KEY`	Signs session cookies. Leakage permits forging an authenticated session, including an administrator session.	Confidentiality
A3	Database credentials (`.env`)	Direct read/write access to every table, bypassing application controls.	Confidentiality
A4	Movement records (who was in which room, when)	Core integrity asset and personal location data, so a privacy asset as well.	Integrity, Confidentiality
A5	Audit log	Provides accountability; only useful if complete, attributable, and tamper-evident.	Integrity, Availability
A6	User and role table	Controls authorization. Whoever edits it controls access to everything else.	Integrity
A7	Session cookies and session rows	Bearer credentials for authenticated access; session hashes enable revocation.	Confidentiality
A8	PostgreSQL database and `pgdata` volume	Aggregates A1, A4, A5, A6; the host volume is a separate exposure path.	All three
A9	Flask application and container image	Enforces every security control; code integrity determines whether controls hold.	Integrity, Availability
A10	Third-party dependencies and base images	Supply-chain exposure through `requirements.txt`, `python:3.12-slim`, `postgres:16`.	Integrity
A11	`schema.sql` and `seed.sql`	Determine default accounts, constraints, and the initial security posture.	Integrity

3. Threat Actors

Actor	Type	Motivation	Capability
Unauthenticated outsider	External	Account takeover, data theft, vandalism	Scripts, credential lists, automated scanners, public exploit code
Malicious guest	External, authenticated	Falsify their own or others' movements; observe where other people are	Valid low-privilege account; can modify requests
Malicious or careless employee	Internal	Conceal their own movements; snoop on guests	Elevated read access and operational knowledge of the system
Rogue administrator	Internal, privileged	Alter records or erase evidence	Full application-level control, including user and role management
"Break-it" competitor team	External	Find exploitable bugs for points	Source-code access and dedicated time — treated as the most capable actor
Supply-chain attacker	External	Compromise via a dependency or base image	Malicious package, typosquat, or tampered image
Host or developer-machine compromise	Internal or external	Steal `.env`, the DB volume, or the signing key	Filesystem or process access on the development or host machine

The break-it phase assumes the attacker has the source code, so "security by obscurity" is not credited anywhere in this model.

4. System Decomposition and DFD

Components

Component	Responsibility	Location
User interface	Jinja2 templates, static CSS and images; no inline script or style under the current CSP.	`src/frontend/templates/`
Authentication module	Login, logout, Argon2id verification, lockout, session creation and revocation.	`src/backend/auth.py`
Authorization layer	Deny-by-default request gate and `@require_role` checks, all server-side.	`src/backend/roles.py`
API endpoints	HTTP routes for views, visits, JSON queries, health, and the admin placeholder.	`src/backend/app.py`
Business logic	Append-only event recording; current state derived from the latest event.	`src/backend/gallery_store.py`
Data access layer	SQLAlchemy Core with bound parameters; no string-built SQL.	`auth_store.py`, `gallery_store.py`
Database	PostgreSQL 16 with typed roles, foreign keys, and indexes.	`database/schema.sql`
Audit logging	Authentication events with actor, target, outcome, and source IP.	`auth_store.write_audit`
Configuration and secrets	`.env` values injected as environment variables.	`.env`, `docker-compose.yml`
External services	Google Fonts stylesheet referenced by the login and layout templates.	`layout.html`, `login.html`

Endpoints (as built)

Method and path	Purpose	Authorization	Status
`GET /`	Home page with room displays	Public	Implemented
`GET /rooms`	Room list	Signed-in user	Template missing
`GET /rooms/<room_id>`	Room detail, artworks, current visitors	Signed-in user	Implemented
`POST /rooms/<room_id>/visit`	Record `enter` or `leave`	Signed-in; employee/admin may pass `user_id`	Implemented
`GET /api/rooms`	JSON room list	Signed-in user	Implemented
`GET /api/whoami`	Current username and role	Signed-in user	Implemented
`GET /health`	Database connectivity check	Public	Implemented
`GET /admin`	Administrator area	Administrator	Placeholder
`GET /history`, `GET /status`	Personal history; current location	Signed-in user	Planned
`/admin/users`	User and role management	Administrator	Planned

Data Flow Diagram — Level 0 (context)

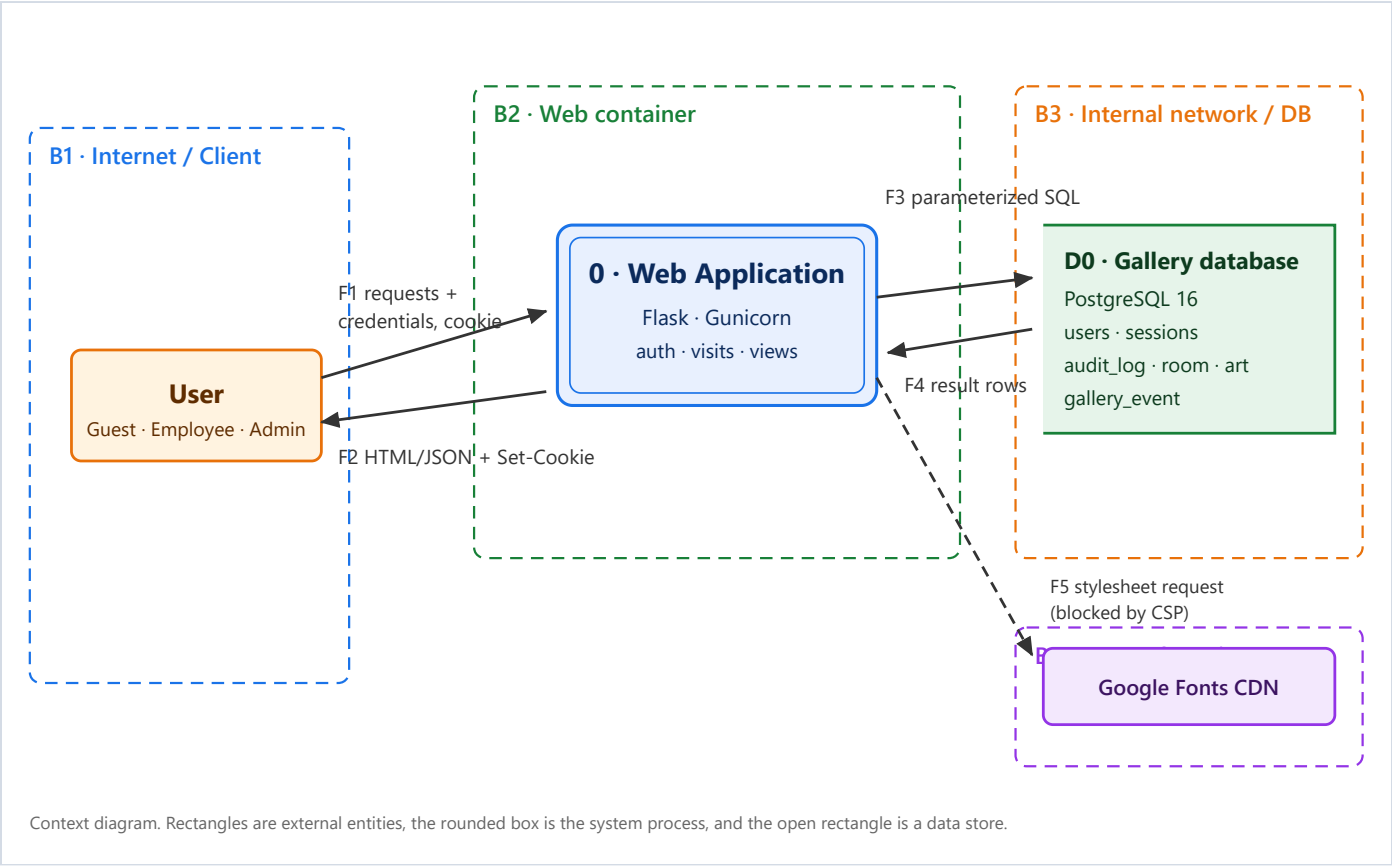


Figure 1. Context diagram: the user, the web application, the gallery database, and the external font service across four trust boundaries.

Data Flow Diagram — Level 1

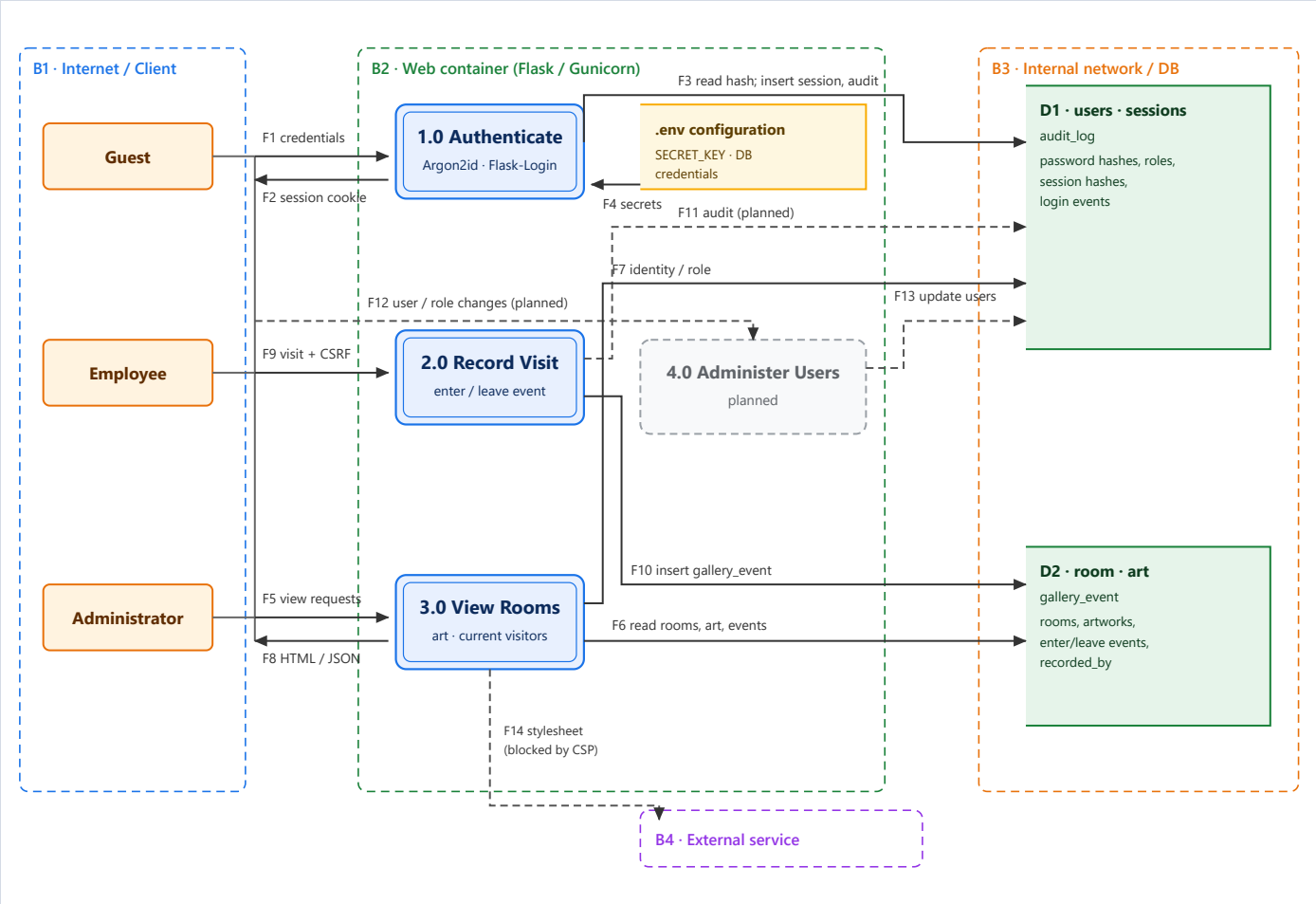


Figure 2. Level 1 decomposition. Processes 1.0–3.0 are implemented; 4.0 is planned. Numbered flows are mapped to threats in Section 5.

Trust boundaries

Boundary	Where it sits	What crosses it
B1	Internet / client to the web container	Credentials, CSRF tokens, session cookies, room identifiers, visit actions. All untrusted input enters here.
B2	Inside the web container	Application code, templates, secrets loaded from `.env`, authorization decisions.
B3	Web container to the database container over the internal Docker network	Parameterized SQL and result rows; no host port is published for the database.
B4	Web container to the external font service	A stylesheet request. Currently blocked by `default-src 'self'`; if allowed, it would disclose visitor IP and referrer.

Flow inventory

Flow	Source → destination	Data	Relevant threats
F1	Guest / Employee / Admin → 1.0	Username, password, CSRF token	T1, T3, T4, T26
F2	1.0 → user	Signed session cookie	T2, T3, T6
F3	1.0 ↔ D1	Password hash, session row, audit row	T2, T21, T22
F4	`.env` → 1.0 (startup)	`SECRET_KEY`, DB credentials	T2, T25
F5	Roles → 3.0	Room identifiers, page requests	T7, T11, T15
F6	3.0 → D2	Room, artwork, and event queries	T17, T20
F7	3.0 → D1	Identity and role lookup	T7, T17

Flow	Source → destination	Data	Relevant threats
F8	3.0 → user	Auto-escaped HTML / JSON	T29
F9	Roles → 2.0	`action`, `room_id`, optional `user_id`, CSRF token	T8, T10, T14, T15, T16
F10	2.0 → D2	`gallery_event` insert	T14, T16, T21
F11	2.0 → D1	Audit insert for delegated actions (planned)	T21, T22, T23
F12	Admin → 4.0	User and role changes (planned)	T7, T9
F13	4.0 → D1	User/role updates (planned)	T7, T9, T22
F14	3.0 → font CDN	Stylesheet request, referrer and IP	T30, T28

Assumptions

- TLS is terminated by a reverse proxy in deployment; the local Compose stack serves HTTP on `127.0.0.1:8000`.
- The break-it team has the source code and valid low-privilege accounts.
- `.env` is not committed and is not copied into the image; the Docker build context still includes it.
- No self-registration exists; accounts are provisioned by seed data or an administrator.
- All timestamps are generated by PostgreSQL `now()`, never by the client.
- Administrative features and personal history are planned, not present, and are modeled as such.

5. STRIDE Threat Identification

Each component was reviewed against all six STRIDE categories. Threats are identified by walking the numbered flows in Figure 2 and asking each STRIDE question at every boundary crossing. A status is attached to every threat because several controls are already implemented; reporting them as open would overstate the residual risk and would not survive a code review.

5.1 Authentication

ID	STRIDE	Threat	Status
T1	Spoofing	Credential stuffing or brute force against `POST /login`.	**Implemented**
T2	Spoofing	Session forgery if `SECRET_KEY` is weak, default, or leaked.	**Partial**
T3	Spoofing	Session hijacking when cookies lack `Secure` or when `FORCE_HTTPS` is off in deployment.	**Partial**
T4	Info Disclosure	Username enumeration through different error messages or response timing.	**Implemented**
T5	Spoofing	Default or weak accounts left in `seed.sql`.	**Open**
T6	Spoofing	Session fixation: the session identifier is not rotated at login and old sessions survive logout.	**Implemented**

T4 is mitigated by a dummy Argon2 verification for unknown users plus one generic error message (`auth.py:63-68`, `auth.py:84-86`). T6 is mitigated by `session.clear()` before login, a fresh server-side token, and revocation on logout (`auth.py:124-127`, `auth_store.py:96-103`).

5.2 Authorization and API endpoints

ID	STRIDE	Threat	Status
T7	Elevation of Privilege	A route lacks a role check — for example a new endpoint, or `/admin/*` as it grows.	**Implemented** (residual for new routes)
T8	Elevation / Tampering	An employee submits another user's ID to `/rooms/<id>/visit`, recording a movement that person never made.	**Partial**
T9	Elevation of Privilege	Mass assignment: a `role` field in a future profile or registration form is accepted unfiltered.	**Planned**
T10	Tampering	Cross-site request forgery against state-changing routes, such as a forced visit or a future role change.	**Implemented**
T11	Info Disclosure	Any signed-in user can see other people's usernames in the current-visitor list for every room.	**Open**
T12	Info Disclosure	Verbose errors, debug mode, or stack traces exposing internals.	**Implemented**
T13	Info Disclosure	`/health` reveals database version, hostnames, or connection errors.	**Implemented**
T30	Info Disclosure	External Google Fonts are blocked by `default-src 'self'`, breaking styling; if the CSP is relaxed, the request discloses visitor IP and referrer to a third party.	**Open**

T7 is covered by the deny-by-default gate in `roles.py:26-38` plus `@require_role` (`roles.py:8-23`). The residual risk is a future route registered without a role and accidentally added to the public set. T8 exists because `app.py:83-89` accepts `user_id` from employees and administrators; guests are forced to `current_user.id`.

5.3 Business logic (movement recording)

ID	STRIDE	Threat	Status
T14	Tampering	Logically impossible events: entering a room without being in the gallery, being in two rooms at once, leaving a room never entered.	**Open**

T15	Tampering	Client-supplied timestamps or room identifiers trusted without validation.	**Partial**
T16	Tampering	Race conditions: concurrent requests create inconsistent visit state (TOCTOU).	**Open** (low exploitability)

`record_event` validates only that the action is `enter` or `leave` and then inserts (`gallery_store.py:87-96`). There is no gallery-level state and no uniqueness constraint preventing a user from being "inside" two rooms. Room IDs are validated for existence (`app.py:90`) and timestamps come from the database, so T15 is partially mitigated.

5.4 Data access layer and database

ID	STRIDE	Threat	Status
T17	Tampering / Info Disclosure	SQL injection through a raw or string-built query, including dynamic `ORDER BY` or `LIKE` filters.	**Implemented**
T18	Elevation of Privilege	The application connects as the PostgreSQL superuser/owner, so any injection becomes total compromise.	**Open**
T19	Info Disclosure	The `pgdata` volume or database backups are readable on the host.	**Open**
T20	Denial of Service	Connection-pool exhaustion or unbounded queries, for example a history endpoint with no pagination.	**Partial**

Every query in `auth_store.py` and `gallery_store.py` uses bound parameters. `docker-compose.yml` sets `POSTGRES_USER` as the database owner, so T18 is a real least-privilege gap.

5.5 Audit logging

ID	STRIDE	Threat	Status
T21	Repudiation	A user denies an action because the record lacks a server-side actor ID, timestamp, or source IP.	**Partial**
T22	Repudiation / Tampering	An administrator, or the application's database user, updates or deletes audit rows to erase evidence.	**Open**
T23	Tampering	Log injection: CR, LF, or control characters in usernames or room names forge log lines.	**Partial**
T24	Info Disclosure	Passwords, session cookies, or CSRF tokens written to logs.	**Implemented**

Only LOGIN and LOGOUT call `write_audit` (`auth.py:80-86` , `auth.py:130-139`). Visit events are stored in `gallery_event` with `recorded_by` but are not copied into `audit_log` , and neither table is protected against `UPDATE` or `DELETE` by the application role.

5.6 Platform, secrets, and dependencies

ID	STRIDE	Threat	Status
T25	Info Disclosure	`.env` committed to git, baked into the image, or printed in logs.	**Partial**
T26	Denial of Service	Login flood: Argon2 is deliberately expensive, and the limiter uses in-memory storage per Gunicorn worker, so the effective limit is three times the configured value.	**Partial**
T27	Elevation of Privilege	The container runs as root, so remote code execution becomes container takeover.	**Implemented**
T28	Tampering	A vulnerable or malicious dependency, or an unpinned base image.	**Partial**
T29	Info Disclosure	Stored XSS through usernames or room names.	**Implemented**

T27 is mitigated by `USER appuser` (`Dockerfile:17`). T29 is mitigated by Jinja auto-escaping and a strict CSP; the same CSP is what blocks the external font request in T30. T26 is only partially mitigated: `10 per minute` is configured (`auth.py:90`) with `memory://` storage (`auth.py:152`) across three workers (`Dockerfile:20`).

6. Risk Analysis and Prioritization

Method. Likelihood (L) and impact (I) are each rated 1–3. Risk is L × I, where 7–9 is **High**, 4–6 is **Medium**, and 1–3 is **Low**. Likelihood reflects exposure, ease of exploitation, and the break-it team's source-code access. Impact reflects damage to assets A1–A11 across confidentiality, integrity, and availability.

Two Medium threats, **T2** and **T22**, are marked *priority Medium*. Their score stays at 6 under the stated method, but either one defeats every other control — forged administrator sessions, or no trustworthy record of events — so they are mitigated in Section 7 alongside the High threats. There is no "Medium-High" band; the scale remains three levels.

ID	Threat	L	I	Score	Severity
T7	Missing role check on a route	3	3	9	**High**
T8	Employee records a visit for another user	3	3	9	**High**
T14	Impossible movement states	3	3	9	**High**
T1	Brute force / credential stuffing	3	2	6	**Medium**
T2	Session forgery via `SECRET_KEY`	2	3	6	**Medium** (priority)
T9	Mass assignment of role	2	3	6	**Medium**
T11	Cross-user visitor visibility	2	3	6	**Medium**
T12	Verbose errors or debug mode	2	3	6	**Medium**
T15	Trusted client timestamps or room IDs	3	2	6	**Medium**
T17	SQL injection	2	3	6	**Medium**
T18	Over-privileged database user	2	3	6	**Medium**
T19	Database volume or backups readable on host	2	3	6	**Medium**
T22	Audit log edited or deleted	2	3	6	**Medium** (priority)
T25	Secrets exposed	2	3	6	**Medium**
T26	Login DoS / limiter bypass	3	2	6	**Medium**
T3	Session hijacking	2	2	4	**Medium**
T5	Default seed accounts	2	2	4	**Medium**
T16	Race conditions	2	2	4	**Medium**
T20	Unbounded queries / pool exhaustion	2	2	4	**Medium**
T21	Repudiation of events	2	2	4	**Medium**
T6	Session fixation	1	3	3	**Low**
T10	CSRF	1	3	3	**Low**
T27	Container runs as root	1	3	3	**Low**
T28	Vulnerable or malicious dependency	1	3	3	**Low**
T29	Stored XSS	1	3	3	**Low**
T13	`/health` information disclosure	2	1	2	**Low**
T23	Log injection	2	1	2	**Low**
T24	Secrets in logs	1	2	2	**Low**
T30	External font dependency / CSP conflict	2	1	2	**Low**

ID	Threat	L	I	Score	Severity
T4	Username enumeration	1	1	1	Low

Risk matrix

Likelihood ↓ / Impact →	Low (1)	Medium (2)	High (3)
High (3)	—	T1, T15, T26	**T7, T8, T14**
Medium (2)	T13, T23, T30	T3, T5, T16, T20, T21	T2, T9, T11, T12, T17, T18, T19, T22, T25
Low (1)	T4	T24	T6, T10, T27, T28, T29

All 30 threats appear exactly once. The matrix cell counts are 3 + 3 + 9 + 5 + 3 + 5 + 1 + 1 = 30.

Rationale for the highest-ranked threats

- **T7 and T8** require no special tooling: a valid account and one modified request. The break-it phase explicitly rewards this class of bug. The current deny-by-default gate protects existing routes, but a single future route registered without a role reopens T7.
- **T14** is the application-specific integrity flaw. The schema stores events but does not constrain them, so a guest can create a state no legitimate traversal could produce. This directly corrupts asset A4 and undermines every downstream occupancy view.
- **T2 and T22** are trust anchors. A forged signing key or a mutable audit log invalidates authentication and accountability respectively, regardless of how well the other controls are implemented. Their likelihood is Medium because both require a leak or privileged access, but their blast radius justifies priority treatment.
- The classic injection family (T10, T17, T29) is deliberately ranked Low: parameterized SQL, CSRFProtect, auto-escaping, and CSP are already in place and verified in the source.

7. Mitigation Strategies

Each mitigation distinguishes what the repository already does from what still needs to change. Code references point at the current implementation.

7.1 T7 — Missing role check on a route

Already in code: a `before_request` gate requires authentication for every endpoint not explicitly public (`roles.py:26-38`); `@require_role` enforces roles (`roles.py:8-23`); `/admin` is protected (`app.py:116-119`).

- Keep the public-endpoint set minimal and review it in every pull request.
- Add a test that walks `app.url_map` and asserts that each role receives 401 or 403 on every non-public route. This turns a forgotten decorator into a failing test rather than a vulnerability.
- Never infer a role from request data; always load it from the database row.

7.2 T8 — Delegated visit recorded for another user

Already in code: guests cannot set `user_id`; the subject defaults to `current_user.id` (`app.py:81-89`). Employees and administrators may set it, and the action is stored with `recorded_by` (`gallery_store.py:87-96`).

- Restrict delegation to an explicit, role-checked endpoint rather than an optional form field on the guest route.
- Record every delegated event in `audit_log` with actor, subject, room, and IP; today it lands only in `gallery_event`.
- Require a reason or ticket identifier for delegated edits and reject self-delegation where it makes no sense.
- Use non-sequential public identifiers (UUIDs) as defense in depth against enumeration.

7.3 T14, T15, T16 — Movement integrity and concurrency

Already in code: the action is allow-listed, the room must exist, and timestamps are generated by the database (`app.py:90-95`, `gallery_store.py:87-96`).

- Implement the state machine server-side: a user must be inside the gallery before entering a room, may occupy one room at a time, and must leave all rooms before leaving the gallery.
- Enforce the same rules in PostgreSQL with a partial unique index for one open room visit per user and a check that `leave` follows an `enter`.
- Wrap the check-and-insert in a transaction with `SELECT ... FOR UPDATE` on the user row, or use `SERIALIZABLE` isolation, to close the TOCTOU window.
- Reject client-supplied timestamps entirely; keep `now()` as the only source of time.

7.4 T2 — Session forgery through the signing key

Already in code: the application refuses to start without a key (`app.py:45-48`); the signing key is supplied through the environment, not source.

- Refuse to start if the key is shorter than 32 bytes or matches a documented placeholder; generate it with `secrets.token_hex(32)`.
- Support rotation: keep the previous key for verification for one session lifetime, then retire it.
- Keep `SESSION_COOKIE_HTTPONLY=True` and `SAMESITE=Lax` (`auth.py:147-148`) and enable `Secure` plus HSTS in deployment.

7.5 T22 and T21 — Audit integrity and non-repudiation

Already in code: authentication events are written with actor, target, outcome, and source IP (`auth_store.py:106-112`); the target is truncated to 64 characters.

- Give the application a separate database role with `INSERT` only on `audit_log`; revoke `UPDATE` and `DELETE` from it.
- Hash-chain rows: each row stores `SHA-256(previous_hash || row_data)`, and a verification script detects edits and deletions.
- Audit every state change, not just login: visits, delegated visits, and administrative changes, with actor, subject, room, and IP.

- Strip CR, LF, and control characters from logged fields (also mitigates T23).

7.6 T1 and T26 — Brute force and login denial of service

Already in code: Argon2id (`auth.py:22`), five-attempt lockout for fifteen minutes (`auth.py:16-17` , `auth_store.py:56-64`), and `10 per minute` on `POST /login` (`auth.py:90`).

- Move Flask-Limiter to Redis so the limit is shared across the three Gunicorn workers; today it is per-worker (`auth.py:152` , `Dockerfile:20`).
- Rate limit per IP and per username, and add exponential backoff; note that account lockout is itself a denial-of-service vector against a known user, so prefer incremental delay for repeated failures.
- Tune Argon2 to a measured 50–100 ms and add TOTP for administrator accounts.

7.7 T17 and T18 — Injection and database privilege

Already in code: every query is parameterized; no dynamic SQL is present.

- Allow-list any future dynamic identifier such as a sort column; never interpolate it.
- Create a least-privilege `app_user` with `SELECT` , `INSERT` , and required `UPDATE` on specific tables only, with no DDL and no ownership.
- Run `bandit` and `semgrep` in CI to catch string-formatted SQL regressions.

7.8 T25, T12, T27 — Secrets and hardening

Already in code: `.env` is git-ignored (`.gitignore:1`) and untracked; the Dockerfile copies only `src/backend` and `src/frontend` , so the file is not baked into the image; the app runs as `appuser` (`Dockerfile:17`); errors return generic messages (`app.py:127-134`).

- Add a `.dockerignore` so `.env` , `.git` , and test data never enter the build context.
- Add `gitleaks` or `git-secrets` as a pre-commit hook and rotate any key that has ever been shared.
- Keep `DEBUG=False` in production, use a read-only root filesystem where possible, and pin base images by digest.

7.9 Remaining Medium threats

Threat	Recommended change
T3	Terminate TLS in deployment, set `FORCE_HTTPS=true` , and confirm `Secure` and HSTS appear on responses.
T5	Rotate seed credentials, remove documented defaults, and force a password change on first login.
T9	When profiles are added, bind only allow-listed fields; never bind a request body directly to a model or row.
T11	Scope visitor views: guests see a count or their own status; employees see usernames for operational rooms; administrators see all.
T19	Restrict host access to the Docker volume, encrypt backups, and test restore access controls.
T20	Paginate list endpoints, set `statement_timeout` , and size the connection pool to the worker count.

8. Threat Summary Table

ID	Component	STRIDE	Threat	Sev.	Status	Mitigation
T7	Authorization	E	Missing role check on a route	High	Implemented	Deny-by-default gate, role decorator; add route-enumeration test
T8	API	E/T	Employee records a visit for another user	High	Partial	Dedicated delegated endpoint, ownership scope, audit actor and subject
T14	Business logic	T	Impossible movement states	High	Open	Server-side state machine plus database constraints
T1	Auth	S	Brute force / credential stuffing	Medium	Implemented	Argon2id, lockout, per-IP limit; Redis-backed limiter
T2	Auth	S	Session forgery via `SECRET_KEY`	Medium	Partial	Strong random key, startup length check, rotation, cookie flags
T9	API	E	Mass assignment of role	Medium	Planned	Explicit field allow-lists when profiles are added
T11	API	I	Cross-user visitor visibility	Medium	Open	Role-scoped visitor views, privacy-by-default
T12	API	I	Verbose errors or debug mode	Medium	Implemented	Generic handlers, `DEBUG=False`, Gunicorn in production
T15	Business logic	T	Trusted client timestamps or room IDs	Medium	Partial	Server timestamps, room existence check, add state validation
T17	Data layer	T/I	SQL injection	Medium	Implemented	Parameterized queries, identifier allow-list, SAST
T18	Database	E	Over-privileged database user	Medium	Open	Least-privilege `app_user`, no DDL or ownership
T19	Database	I	Database volume or backups readable on host	Medium	Open	Volume access control, encrypted backups
T22	Audit	R/T	Audit log edited or deleted	Medium	Open	INSERT-only role, hash-chained entries, verification script
T25	Platform	I	Secrets exposed	Medium	Partial	`.gitignore`, `.dockerignore`, secret scanning, rotation
T26	Platform	D	Login DoS / limiter bypass	Medium	Partial	Shared Redis limiter, per-user limits, tuned Argon2 cost
T3	Auth	S	Session hijacking	Medium	Partial	Secure and HttpOnly cookies, HSTS, `FORCE_HTTPS=true`
T5	Database	S	Default seed accounts	Medium	Open	Remove seed credentials, force first-login reset
T16	Business logic	T	Race conditions	Medium	Open	Row locks or serializable transactions
T20	Database	D	Unbounded queries / pool exhaustion	Medium	Partial	Pagination, `statement_timeout`, pool sizing
T21	Audit	R	Repudiation of events	Medium	Partial	Unified audit with actor, subject, timestamp, IP
T6	Auth	S	Session fixation	Low	Implemented	`session.clear()`, fresh server-side token, revoke on logout

ID	Component	STRIDE	Threat	Sev.	Status	Mitigation
T10	API	T	CSRF	Low	Implemented	`CSRFProtect` on all state-changing routes, tokens in forms
T27	Platform	E	Container runs as root	Low	Implemented	`USER appuser`, read-only filesystem where possible
T28	Platform	T	Vulnerable or malicious dependency	Low	Partial	Pin digests, `pip-audit`, Dependabot
T29	UI	I	Stored XSS	Low	Implemented	Jinja auto-escape and strict CSP; no inline script
T13	API	I	`/health` information disclosure	Low	Implemented	Return only `{"status":"ok"}`; log details server-side
T23	Audit	T	Log injection	Low	Partial	Strip CR, LF, and control characters from logged fields
T24	Audit	I	Secrets in logs	Low	Implemented	No credential or token logging; keep it that way in new code
T30	UI / Platform	I	External font dependency and CSP conflict	Low	Open	Self-host the fonts or remove the external links
T4	Auth	I	Username enumeration	Low	Implemented	Uniform error and dummy hash verification for unknown users

9. Conclusion

The project starts from a stronger baseline than the original six-row STRIDE table suggested. Argon2id with a dummy verification path, parameterized queries, global CSRF protection, Jinja auto-escaping, a strict CSP, a non-root container, and an internal-only database remove or sharply reduce the classic injection family. Those controls are verified in source, not merely claimed.

The remaining risk is concentrated in application-specific logic and trust anchors:

1. **Delegated visit forgery (T8)** — an employee can record a movement for any user through an optional form field.
2. **Cross-user visitor visibility (T11)** — every signed-in user can see other people's usernames in every room.
3. **No movement state machine (T14–T16)** — events are appended without enforcing physically possible transitions, and concurrent requests can interleave.
4. **Incomplete, mutable audit trail (T21–T22)** — only login and logout are audited, and rows can be edited or deleted by the application's database user.
5. **Deployment-dependent session protection (T2–T3, T26)** — cookie `Secure`, HSTS, and a shared rate-limit store all depend on settings that are off by default.
6. **Over-privileged database account (T18)** — the application connects as the database owner.

Most fixes are local and fit the existing Flask and PostgreSQL architecture: a state machine backed by a partial unique index, an append-only audit role, a dedicated delegated-visit route, a Redis-backed limiter, and a least-privilege `app_user`. None requires a redesign, and each can be verified with a focused security test.

10. References

- OWASP Foundation. *OWASP Threat Modeling* and the *Threat Modeling Cheat Sheet*.
- Microsoft. *The STRIDE Threat Model* and *Threat Modeling: Uncover Security Design Flaws Using the STRIDE Approach*.
- OWASP. *Application Security Verification Standard (ASVS)* and *Top 10* — authentication, access control, and logging categories.
- MITRE. *CWE Top 25* — CWE-639 (IDOR), CWE-89 (SQL injection), CWE-352 (CSRF), CWE-778 (insufficient logging).
- Shostack, A. *Threat Modeling: Designing for Security*. Wiley, 2014.

Appendix A — As-Built vs. Target Design

Capability	As built	Target / note
Login, lockout, sessions	Yes	Add TOTP for administrators and a startup key-length check.
Room enter/leave	Yes	Add server-side state validation and DB constraints.
Gallery-level enter/leave	No	Designed but absent; requires a gallery table and events.
Personal movement history	No	No `/history` endpoint; add pagination and ownership checks.
Admin user/role management	No	`/admin` returns JSON; build forms with allow-listed fields.
Delegated visit recording	Partial	Works via optional `user_id`; move to a dedicated audited endpoint.
Audit of all state changes	Partial	Auth events only; visits are in `gallery_event`.
Rate limiting	Partial	Login only, in-memory, per worker.
Least-privilege database role	No	App connects as the owner; create `app_user`.
Test suite runnable	No	`test/security/test_auth.py` imports a missing `fakes.py`.

Appendix B — Evidence Index

Claim	Location
Argon2id and dummy verification	`src/backend/auth.py:22`, `auth.py:63-68`
Lockout (5 attempts / 15 minutes)	`auth.py:16-17`, `auth_store.py:56-64`
Login rate limit	`auth.py:90`; in-memory storage `auth.py:152`
Session rotation and revocation	`auth.py:124-127`, `auth_store.py:75-103`
Cookie flags	`auth.py:147-151`
CSRF protection	`auth.py:28`, `auth.py:156`; template tokens in `login.html:29`, `layout.html:28`, `room.html:24`
CSP, frame deny, HSTS when enabled	`app.py:50-52`
Deny-by-default authorization	`roles.py:26-38`; `app.py:16`
Role decorator	`roles.py:8-23`; `app.py:116-119`
Delegated visit via `user_id`	`app.py:77-96`
Visit validation and insert	`gallery_store.py:87-96`
Generic error handling	`app.py:121-134`
Parameterized queries	`auth_store.py`, `gallery_store.py` (all queries)
Audit write	`auth_store.py:106-112`; callers `auth.py:80-86`
Schema and constraints	`database/schema.sql:12-81`
Non-root container	`src/backend/Dockerfile:7`, `Dockerfile:17`
Three Gunicorn workers	`Dockerfile:20`
Database not exposed to host	`docker-compose.yml:19-21`
Local binding and HTTP	`docker-compose.yml:31-33`
Container hardening	`docker-compose.yml:37-40`
`.env` ignored by git	`.gitignore:1`
External font references	`layout.html:11-14`, `login.html:9-12`
Missing test fixture	`test/security/test_auth.py:18` (imports `fakes`)
Missing room list template	`app.py:60-62` renders `rooms.html`, which is absent

Prepared for the Secure Programming threat modeling assignment. The report describes the repository at commit `f60f642`; planned features are labeled as such and were not credited as implemented controls.